

Deep Learning – 1RT720

Report for Hand-in Assignment 1

Rafael Proença

February 7, 2026

1 Report

Exercise 1. Implement a multi-layer neural network

In Exercise 1, I implement a fully connected neural network for multi-class classification of MNIST digits using only NumPy. The network supports an arbitrary number of layers specified as a list (e.g., [784, 128, 64, 10]). I use the rows-as-data-points convention throughout: $\mathbf{X}$ has shape $(N, 784)$ and $\mathbf{Y}$ has shape $(N, 10)$ in one-hot encoding.

Key design choices:

- **He initialization** for weights ($W_k \sim \mathcal{N}(0, \sqrt{2/n_{\text{in}}})$) and zeros for biases, which is important for stable training with ReLU activations.
- **Numerically stable softmax and cross-entropy**: I subtract the row-wise maximum from the logits before computing exponentials, preventing overflow.
- **Fully vectorized**: all forward and backward computations use matrix operations (@) with no loops over data points or hidden units.
- **Mini-batch training**: each epoch shuffles the data and partitions it into mini-batches of a specified size.

Data is loaded using the provided `load_mnist` function from `utils.py`. The full code is available in Appendix A.

Exercise 2a. Evaluate a linear model

I train a linear model (zero hidden layers, architecture [784, 10]) on the MNIST dataset using mini-batch gradient descent with batch size 64, learning rate 0.1, and 50 epochs. As required by the assignment, the training cost and accuracy are evaluated on the current (last) mini-batch at each epoch rather than the full training set.

The model reaches a test accuracy of 92.69%, exceeding the 90% target. In Figure 1, the test cost and accuracy curves are smooth since they are evaluated on the full 10 000-sample test set, while the training curves show characteristic oscillations because they are computed on a single mini-batch of 64 samples—a different random subset each epoch.

Exercise 2: Linear Model (No Hidden Layers)

| Batch size:64 | Learning rate:0.1 | Number of Epochs:50 | Training time: 0 min 19.1 sec |

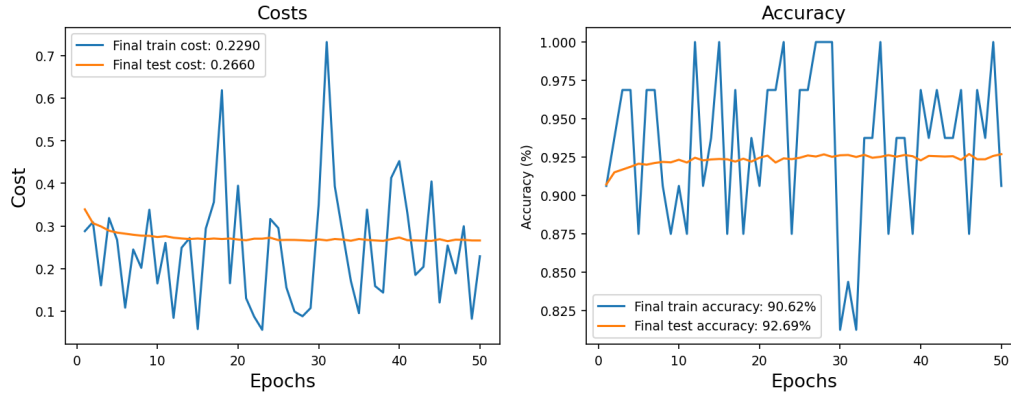


Figure 1: Exercise 2a: Cost and accuracy for the linear model ($[784, 10]$) with batch size 64, learning rate 0.1, over 50 epochs. The test accuracy converges to 92.69%. The training curves oscillate due to mini-batch evaluation, while the test curves are smooth, indicating stable convergence.

Exercise 2b. Plot weight matrices

I extract each of the ten columns of the weight matrix W (shape 784×10), reshape them to 28×28 , and visualize them as images in Figure 2. Each image represents the learned “template” for the corresponding digit class. In the linear model, classification is performed by computing the dot product between the input image and each template, so bright regions indicate pixels that increase the score for that digit, while dark regions decrease it.

Some digits (e.g., 0, 1) produce clearly recognizable templates because their shapes are relatively consistent across samples. Others (e.g., 5, 8, 9) are noisier because those digits exhibit greater variation in handwriting style, making it harder for a single linear template to capture them.

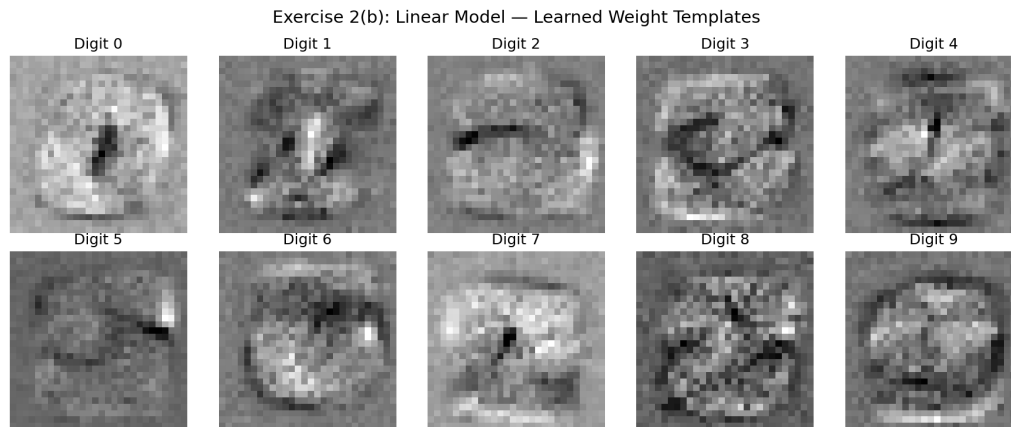


Figure 2: Exercise 2b: Learned weight templates for the linear model. Each 28×28 image corresponds to one digit class (0–9). Bright pixels indicate positive weight (supporting that class), dark pixels indicate negative weight (opposing it). Digits with consistent shapes (e.g., 0, 1) produce clearer templates.

Exercise 3. Evaluate your multi-layer neural network

I train a multi-layer network with architecture $[784, 128, 64, 10]$ (two hidden layers) using both ReLU and sigmoid activation functions. Both models use batch size 64 and 50 epochs. For ReLU I use learning

rate 0.1, and for sigmoid I use 0.5 (sigmoid benefits from a larger step size since its gradients are smaller in magnitude).

The ReLU model achieves 98.04% test accuracy (Figure 3) and the sigmoid model achieves a comparable result (Figure 4). Both exceed the 95% target. The training cost curves are near zero because the model classifies nearly every 64-sample mini-batch perfectly after about 10 epochs—this is expected behavior with mini-batch evaluation on a well-fitting model.

I chose the [784, 128, 64, 10] architecture as a reasonable starting point with decreasing layer widths. Both the learning rates and the architecture were tuned by a small number of experiments.

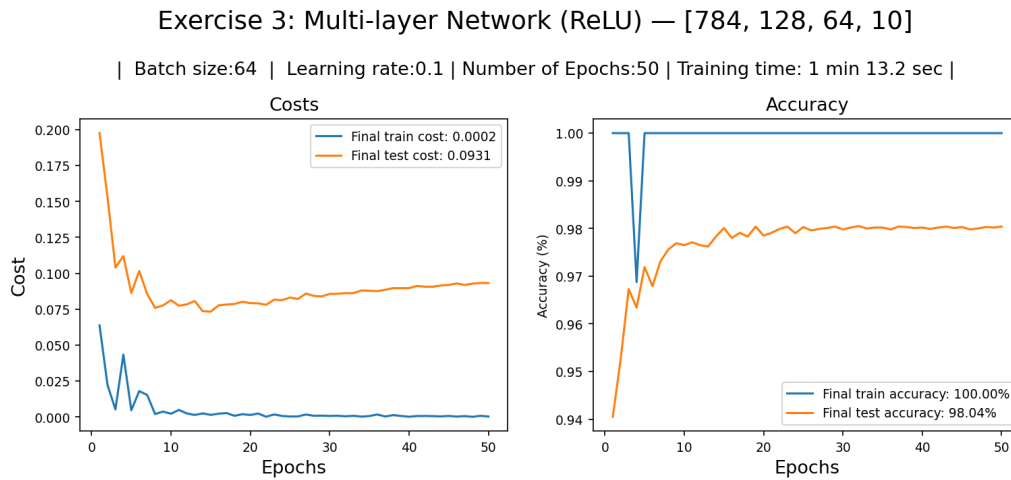


Figure 3: Exercise 3: Cost and accuracy for the multi-layer network with ReLU activation ([784, 128, 64, 10], batch size 64, learning rate 0.1, 50 epochs). Test accuracy reaches 98.04%. The training cost drops to near zero as the model perfectly classifies each mini-batch.

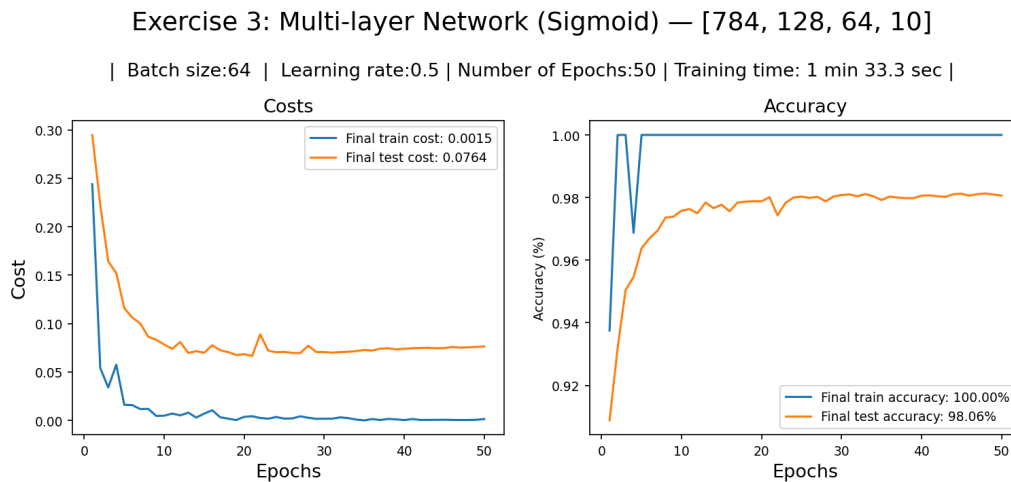


Figure 4: Exercise 3: Cost and accuracy for the multi-layer network with sigmoid activation ([784, 128, 64, 10], batch size 64, learning rate 0.5, 50 epochs). The sigmoid model achieves comparable performance to ReLU but requires a higher learning rate to compensate for smaller gradient magnitudes.

Use of generative AI

I used Gemini to assist with debugging vectorized NumPy code and structuring functions. I also used it to help draft the report. All AI-generated content was reviewed and tested by me.

- ☒ The report generally and transparently details all uses of AI-tools, e.g., as a minimum by explaining how it was used and to what extent (see the acknowledgment for an example).
- ☒ Text or code proposed or modified by an AI has been checked for correctness and that the meaning is as intended.
- ☒ I take full responsibility for the whole text.

A Code

```
1  # -*- coding: utf-8 -*-
2  """hand_in_1.ipynb
3
4  Automatically generated by Colab.
5
6  Original file is located at
7      https://colab.research.google.com/drive/1Zu3VNmGZYWmCIm9_wDM_Wbv6VW1BgFd7
8
9  # Deep Learning 2026: Hand-in Assignment 1
10 ## Fully Connected Neural Network for MNIST Classification
11
12 **Convention:** rows = data points
13 - `X`: (N, 784) | each row is one flattened image
14 - `Y`: (N, 10) | one-hot encoded labels
15 - `W_k`: (n_in, n_out), `b_k`: (1, n_out)
16 - `Z_k = A_{k-1} @ W_k + b_k` (pre-activation)
17 - `A_k = activation(Z_k)` (post-activation)
18 """
19
20 import numpy as np
21 import time
22 from pathlib import Path
23 import os, sys
24
25 # Ensure working directory contains MNIST/ and utils.py
26 _start_dir = Path.cwd().resolve()
27 for _p in [_start_dir] + list(_start_dir.parents):
28     if (_p / 'MNIST').exists() and (_p / 'utils.py').exists():
29         os.chdir(_p)
30         if str(_p) not in sys.path:
31             sys.path.insert(0, str(_p))
32         break
33
34 from matplotlib import pyplot as plt
35 from utils import load_mnist, training_curve_plot
36
37 """## Load Data"""
38
39 X_train, Y_train, X_test, Y_test = load_mnist()
40 print(f"X_train: {X_train.shape}, Y_train: {Y_train.shape}")
41 print(f"X_test: {X_test.shape}, Y_test: {Y_test.shape}")
42
43 """## Exercise 1: Neural Network Implementation
44
45 ### 1. Initialization (He initialization)
46 """
47
48 def initialize(model):
49     """
50     Initialize parameters for a fully connected network using He initialization.
51
52     param model: list of int, number of units in each layer, e.g. [784, 128, 64, 10]
53     return: list of dicts, each with keys 'W' (weight matrix) and 'b' (bias vector)
54     """
55     params = []
56     for k in range(len(model) - 1):
57         n_in, n_out = model[k], model[k + 1]
58         W = np.random.randn(n_in, n_out) * np.sqrt(2.0 / n_in) # He initialization
59         b = np.zeros((1, n_out))
60         params.append({'W': W, 'b': b})
61     return params
62
63 """## 2. Activation Functions and Derivatives"""
64
65 def sigmoid(Z):
66     """Sigmoid activation:  $\sigma(z) = 1 / (1 + \exp(-z))$ ."""
67     return 1.0 / (1.0 + np.exp(-Z))
68
69 def sigmoid_derivative(Z):
70     """Derivative of sigmoid w.r.t. its input:  $\sigma(z) * (1 - \sigma(z))$ ."""
71     s = sigmoid(Z)
```

```

72     return s * (1.0 - s)
73
74 def relu(Z):
75     """ReLU activation:  $a(z) = \max(0, z)$ ."""
76     return np.maximum(0, Z)
77
78 def relu_derivative(Z):
79     """Derivative of ReLU w.r.t. its input: 1 if  $z > 0$ , else 0."""
80     return np.where(Z > 0, 1.0, 0.0)
81
82 """### 3. Forward Propagation"""
83
84 def forward_pass(X, params, activation_fn='relu'):
85     """
86     Forward propagation through all layers.
87     Hidden layers use the chosen activation; the output layer is linear
88     (softmax is applied in the cost function for numerical stability).
89
90     param X: input data, shape (N, D_in)
91     param params: list of dicts with 'W' and 'b' for each layer
92     param activation_fn: 'relu' or 'sigmoid' for hidden layers
93     return: dict with 'Z' (pre-activations) and 'A' (activations) for all layers
94     """
95     activation = relu if activation_fn == 'relu' else sigmoid
96     K = len(params)
97
98     A = [None] * (K + 1)
99     Z = [None] * K
100    A[0] = X
101
102    for k in range(K):
103        Z[k] = A[k] @ params[k]['W'] + params[k]['b']
104        if k < K - 1:
105            A[k + 1] = activation(Z[k])
106        else:
107            A[k + 1] = Z[k] # linear output layer
108
109    return {'Z': Z, 'A': A}
110
111 """### 4. Softmax, Cost, and Gradient
112
113 Cross-entropy loss combined with softmax (Eq. 2 from assignment):
114  $\mathcal{L} \ell_{\ell} = -\log \left( \sum_{k=1}^{D_o} \exp(f_{ik}) \right)^{-1} - \sum_{k=1}^{D_o} \tilde{y}_{ik} f_{ik}$ ,  $\text{quad } L$ 
115  $\hookrightarrow = \frac{1}{N} \sum_{i=1}^N \ell_{\ell}$ 
116
117 Gradient:  $\mathcal{L} \frac{\partial}{\partial f} = \frac{1}{N} (\text{softmax}(f) - \tilde{y}) \mathcal{L}$ 
118 """
119
120 def softmax(F):
121     """Numerically stable softmax."""
122     F_stable = F - np.max(F, axis=1, keepdims=True)
123     exp_F = np.exp(F_stable)
124     return exp_F / np.sum(exp_F, axis=1, keepdims=True)
125
126 def compute_cost(F, Y_onehot):
127     """Cross-entropy cost combined with softmax (numerically stable)."""
128     N = F.shape[0]
129     F_stable = F - np.max(F, axis=1, keepdims=True)
130     log_sum_exp = np.log(np.sum(np.exp(F_stable), axis=1))
131     correct_class_score = np.sum(F_stable * Y_onehot, axis=1)
132     return np.mean(log_sum_exp - correct_class_score)
133
134 def d_cost_d_output(F, Y_onehot):
135     """Derivative of cross-entropy cost w.r.t. last-layer linear output."""
136     N = F.shape[0]
137     return (softmax(F) - Y_onehot) / N
138
139 """### 5. Backward Propagation"""
140
141 def backward_pass(params, cache, Y_onehot, activation_fn='relu'):
142     """
143     Backward propagation through all layers.
144     Returns gradients for all parameters.
145     """
146     act_deriv = relu_derivative if activation_fn == 'relu' else sigmoid_derivative

```

```

146     Z, A = cache['Z'], cache['A']
147     K = len(params)
148     grads = [None] * K
149
150     # Start from the output
151     dA = d_cost_d_output(A[K], Y_onehot)
152
153     for k in reversed(range(K)):
154         if k == K - 1:
155             dZ = dA # output layer: no activation applied
156         else:
157             dZ = dA * act_deriv(Z[k]) # hidden layer
158
159         grads[k] = {
160             'dW': A[k].T @ dZ,
161             'db': np.sum(dZ, axis=0, keepdims=True)
162         }
163
164         if k > 0:
165             dA = dZ @ params[k]['W'].T
166
167     return grads
168
169 ##### 6. Parameter Update#####
170
171 def update_parameters(params, grads, learning_rate):
172     """
173     Update parameters using gradient descent.
174
175     param params: list of dicts with 'W' and 'b' for each layer
176     param grads: list of dicts with 'dW' and 'db' for each layer
177     param learning_rate: step size alpha
178     return: updated params
179     """
180     for k in range(len(params)):
181         params[k]['W'] -= learning_rate * grads[k]['dW']
182         params[k]['b'] -= learning_rate * grads[k]['db']
183     return params
184
185 ##### 7. Predict#####
186
187 def predict(X, Y_onehot, params, activation_fn='relu'):
188     """
189     Predict class labels and compute accuracy and cost.
190
191     param X: input data, shape (N, D_in)
192     param Y_onehot: one-hot encoded labels, shape (N, D_out)
193     param params: trained model parameters
194     param activation_fn: 'relu' or 'sigmoid'
195     return: (predictions, accuracy, cost)
196     """
197     cache = forward_pass(X, params, activation_fn)
198     F = cache['A'][-1]
199     cost = compute_cost(F, Y_onehot)
200     predictions = np.argmax(softmax(F), axis=1)
201     true_labels = np.argmax(Y_onehot, axis=1)
202     accuracy = np.mean(predictions == true_labels)
203     return predictions, accuracy, cost
204
205 ##### 8. Mini-batch Generation#####
206
207 def random_mini_batches(X, Y, batch_size):
208     """
209     Randomly partition training data into mini-batches.
210
211     param X: input data, shape (N, D_in)
212     param Y: labels, shape (N, D_out)
213     param batch_size: number of samples per mini-batch
214     return: list of (X_mini, Y_mini) tuples
215     """
216     N = X.shape[0]
217     indices = np.random.permutation(N)
218     X_shuffled, Y_shuffled = X[indices], Y[indices]
219
220     mini_batches = []

```

```

221     num_complete = N // batch_size
222     for i in range(num_complete):
223         s, e = i * batch_size, (i + 1) * batch_size
224         mini_batches.append((X_shuffled[s:e], Y_shuffled[s:e]))
225     if N % batch_size != 0:
226         mini_batches.append((X_shuffled[num_complete * batch_size:],
227                               Y_shuffled[num_complete * batch_size:]))
228     return mini_batches
229
230 """### 9. Model Training"""
231
232 def train_model(X_train, Y_train, model, num_epochs, learning_rate, batch_size,
233                 activation_fn='relu', X_test=None, Y_test=None, eval_every=1):
234     """
235     Train a fully connected neural network with mini-batch gradient descent.
236
237     param X_train: training input, shape (N, D_in)
238     param Y_train: training labels (one-hot), shape (N, D_out)
239     param model: list of int, architecture e.g. [784, 128, 64, 10]
240     param num_epochs: number of training epochs
241     param learning_rate: step size alpha
242     param batch_size: number of samples per mini-batch
243     param activation_fn: 'relu' or 'sigmoid' for hidden layers
244     param X_test: optional test input for monitoring
245     param Y_test: optional test labels for monitoring
246     param eval_every: evaluate every k-th epoch
247     return: (params, history dict, elapsed time in seconds)
248     """
249     params = initialize(model)
250     train_costs, test_costs, train_accs, test_accs = [], [], [], []
251
252     start = time.time()
253     for epoch in range(1, num_epochs + 1):
254         for X_mb, Y_mb in random_mini_batches(X_train, Y_train, batch_size):
255             cache = forward_pass(X_mb, params, activation_fn)
256             grads = backward_pass(params, cache, Y_mb, activation_fn)
257             params = update_parameters(params, grads, learning_rate)
258
259         if epoch % eval_every == 0 or epoch == 1:
260             # Evaluate training cost/accuracy on the last mini-batch (as per assignment)
261             _, tr_acc, tr_cost = predict(X_mb, Y_mb, params, activation_fn)
262             train_costs.append(tr_cost); train_accs.append(tr_acc)
263
264         if X_test is not None:
265             _, te_acc, te_cost = predict(X_test, Y_test, params, activation_fn)
266             test_costs.append(te_cost); test_accs.append(te_acc)
267
268         msg = f"Epoch {epoch:3d}/{num_epochs} | Train: cost={tr_cost:.4f} acc={100*tr_acc:.2f}%"
269         if X_test is not None:
270             msg += f" | Test: cost={te_cost:.4f} acc={100*te_acc:.2f}%"
271         print(msg)
272
273     elapsed = time.time() - start
274     history = {
275         'train_cost': np.array(train_costs), 'test_cost': np.array(test_costs),
276         'train_acc': np.array(train_accs), 'test_acc': np.array(test_accs)
277     }
278     return params, history, elapsed
279
280 """### Exercise 2: Linear Model (0 Hidden Layers)
281
282 A linear model corresponds to `model = [784, 10]` | no hidden layers, just a direct mapping from input
283 ↪ pixels to class logits. I should reach >90% test accuracy.
284 """
285
286 linear_model = [784, 10]
287 lr_linear = 0.1
288 bs_linear = 64
289 epochs_linear = 50
290
291 params_linear, hist_linear, elapsed_linear = train_model(
292     X_train, Y_train,
293     model=linear_model,
294     num_epochs=epochs_linear,
295     learning_rate=lr_linear,

```



```

295     batch_size=bs_linear,
296     activation_fn='relu', # irrelevant: no hidden layers
297     X_test=X_test, Y_test=Y_test,
298 )
299
300 """### Exercise 2(a): Training Curves"""
301
302 training_curve_plot(
303     "Exercise 2: Linear Model (No Hidden Layers)",
304     hist_linear['train_cost'], hist_linear['test_cost'],
305     hist_linear['train_acc'], hist_linear['test_acc'],
306     batch_size=bs_linear, learning_rate=lr_linear,
307     num_epochs=epochs_linear, elapsed=elapsed_linear,
308 )
309 plt.savefig("exercise2_training_curve.png", dpi=150, bbox_inches='tight')
310 plt.show()
311
312 """### Exercise 2(b): Weight Visualization
313
314 Each column of the weight matrix W (shape 784x10) corresponds to one digit class.
315 Reshaped to 28x28, these show the "template" each class is looking for.
316 """
317
318 W = params_linear[0]['W'] # shape (784, 10)
319 fig, axes = plt.subplots(2, 5, figsize=(12, 5))
320 fig.suptitle("Exercise 2(b): Linear Model | Learned Weight Templates", fontsize=14)
321 for digit in range(10):
322     ax = axes[digit // 5, digit % 5]
323     img = W[:, digit].reshape(28, 28)
324     ax.imshow(img, cmap='gray')
325     ax.set_title(f"Digit {digit}")
326     ax.axis('off')
327 plt.tight_layout()
328 plt.savefig("exercise2_weights.png", dpi=150, bbox_inches='tight')
329 plt.show()
330
331 """### Exercise 3: Multi-layer Neural Network
332
333 I train with two hidden layers [784, 128, 64, 10] using both ReLU and Sigmoid activations.
334 Target: ~97-98% test accuracy with ReLU.
335
336 ### ReLU Activation
337 """
338
339 deep_model = [784, 128, 64, 10]
340
341 params_relu, hist_relu, elapsed_relu = train_model(
342     X_train, Y_train,
343     model=deep_model,
344     num_epochs=50,
345     learning_rate=0.1,
346     batch_size=64,
347     activation_fn='relu',
348     X_test=X_test, Y_test=Y_test,
349 )
350
351 training_curve_plot(
352     "Exercise 3: Multi-layer Network (ReLU) | [784, 128, 64, 10]",
353     hist_relu['train_cost'], hist_relu['test_cost'],
354     hist_relu['train_acc'], hist_relu['test_acc'],
355     batch_size=64, learning_rate=0.1,
356     num_epochs=50, elapsed=elapsed_relu,
357 )
358 plt.savefig("exercise3_relu.png", dpi=150, bbox_inches='tight')
359 plt.show()
360
361 """### Sigmoid Activation"""
362
363 params_sig, hist_sig, elapsed_sig = train_model(
364     X_train, Y_train,
365     model=deep_model,
366     num_epochs=50,
367     learning_rate=0.5,
368     batch_size=64,
369     activation_fn='sigmoid',

```

```
370     X_test=X_test, Y_test=Y_test,
371 )
372
373 training_curve_plot(
374     "Exercise 3: Multi-layer Network (Sigmoid) | [784, 128, 64, 10]",
375     hist_sig['train_cost'], hist_sig['test_cost'],
376     hist_sig['train_acc'], hist_sig['test_acc'],
377     batch_size=64, learning_rate=0.5,
378     num_epochs=50, elapsed=elapsed_sig,
379 )
380 plt.savefig("exercise3_sigmoid.png", dpi=150, bbox_inches='tight')
381 plt.show()
```
